

```
squares of residuals
sst = sum(((y_true) - mean(y_true)) .^ 2))
ssr = sum(((y_pred) - y_true) .^ 2))

r_square = 1 - (ssr / sst)

return r_square
end

# Get the r-squared score for training and test datasets
train_r_square = r_squared_score(pred_train, y_train)

println("Training R square score for test sets: ", train_r_square)
```

```
Training R square score for test sets: 0.7341702342769436
```

Figure 3: R-Square score for test data

```
Out[13]: StatsModels.TableRegressionModel{LinearModel{GLM.LmResp{Array{Float64,1}},GLM.DensePredChol{Float64,LinearAlgebra.Cholesky{Float64,Array{Float64,2}}}},Array{Float64,2}}
```

medv ~ 1 + crim + zn + indus + chas + nox + rm + age + dis + rad + tax + ptratio + black + lstat

Coefficients:

|             | Coef.       | Std. Error | t      | Pr(> t ) | Lower 95%  | Upper 95%  |
|-------------|-------------|------------|--------|----------|------------|------------|
| (Intercept) | 36.4595     | 5.10346    | 7.14   | <1e-11   | 26.4322    | 46.4868    |
| crim        | -0.108011   | 0.032865   | -3.29  | 0.0011   | -0.172584  | -0.0434383 |
| zn          | 0.0464205   | 0.0137275  | 3.38   | 0.0008   | 0.0194488  | 0.0733921  |
| indus       | 0.0205586   | 0.0614957  | 0.33   | 0.7383   | -0.100268  | 0.141385   |
| chas        | 2.68673     | 0.86158    | 3.12   | 0.0019   | 0.993904   | 4.37956    |
| nox         | -17.7666    | 3.81974    | -4.65  | <1e-5    | -25.2716   | -10.2616   |
| rm          | 3.80987     | 0.417925   | 9.12   | <1e-17   | 2.98873    | 4.631      |
| age         | 0.000692225 | 0.0132098  | 0.05   | 0.9582   | -0.0252623 | 0.0266468  |
| dis         | -1.47557    | 0.199455   | -7.40  | <1e-12   | -1.86745   | -1.08368   |
| rad         | 0.306049    | 0.0663464  | 4.61   | <1e-5    | 0.175692   | 0.436407   |
| tax         | -0.0123346  | 0.00376054 | -3.28  | 0.0011   | -0.0197233 | -0.0049459 |
| ptratio     | -0.952747   | 0.130827   | -7.28  | <1e-11   | -1.2098    | -0.695699  |
| black       | 0.00931168  | 0.00268596 | 3.47   | 0.0006   | 0.00403431 | 0.0145891  |
| lstat       | -0.524758   | 0.0507153  | -10.35 | <1e-22   | -0.624404  | -0.425113  |

Figure 4: Output of regression model using stats package

Figure 3 shows us the r-square score for the model which can be rounded to about 0.73. In this demonstration, we trained the linear regression model from scratch. Of course, just like Python, you always have the option to use the model directly from the stats package in Julia. That way, you can execute this entire process in four lines of code.

```
df = DataFrame(CSV.File("Boston.csv"))
fm = @formula(medv ~ crim +zn+indus+chas+nox+rm+age+dis+rad+tax+ptratio+black+lstat)
linearRegressor = lm(fm, df)
```

Figure 4 shows us the coefficients that are derived using the linear regression model from the stats package. Let us now calculate the r-square for this model:

```
# R Square value of the model
r2(linearRegressor)
```

From Figure 5, it becomes abundantly clear that it doesn't make much of a difference whether you train data from scratch or use the more convenient approach of using a regression model from the stats package. The difference between both the r-squared scores, i.e., 0.73 and 0.74, is negligible.

In this article, we implemented a linear regression model using the Julia programming language to predict prices in the housing market of Boston, Massachusetts, USA. This article has been written to give you a taste of the Julia programming language. The example used is but a simple demonstration of the power of Julia. If you are coming from a Python background, then you are most certainly going to find many similarities between the two languages in terms of syntax, structure and approach. This is good news as the learning curve is less.

```
Out[14]: 0.7406426641094095
```

Figure 5: The r-square score using stats package

It is my opinion that Julia is a must have on your resume, whether you are a practising or a prospective data scientist.  END

 By: Mir H.S. Quadri

The author is a scholar and researcher in the fields of artificial intelligence and machine learning. He has worked on multiple projects using a wide array of frameworks. He is also a FOSS enthusiast and actively contributes to several open source projects. You can find his works on [codelatte.site](https://codelatte.site).